

1 Uvod

V tem diplomskem delu, bomo raziskali, kaj je linearna logika in kako se jo uporablja v računalništvu. Linearna logiko je prvi predstavil Jean-Yves Girard [2]. Zelo hitro se je izkazala za uporabno, saj zahteva, da smo pozorni na kaj imamo na voljo v danem trenutku, kar bolj odseva realni svet. Zato linearni logiki pravimo tudi *logika stanja*.

2 Naravna dedukcija

V tem poglavju bomo najprej predstavili pravila sklepanja na katerih temelji naravna dedukcija oziroma “standardna” logika in s tem bomo tudi predstavili notacijo, ki bo uporabljena v tem diplomskem delu.

Konjunkcija. Pravilo vpeljave za konjunkcijo pravi, če sta izjavi A in B resnični, potem je tudi izjava A in B resnična. To lahko zapišemo kot

$$\frac{A \quad B}{A \wedge B} \wedge I.$$

Nad črto zapišemo končno mnogo premis in pod črto zapišemo *sklep*. Desno od črte zapišemo ime pravila, ki ga ponavadi okrajšamo. V našem primeru $\wedge I$ predstavlja pravilo vpeljave za konjunkcijo (ang. conjunction introduction). Splošna oblika pravila sklepanja je torej

$$\frac{\mathcal{H}_1 \quad \mathcal{H}_2 \quad \dots \quad \mathcal{H}_n}{\mathcal{S}} \text{ime.}$$

Izjavam $\mathcal{S}, \mathcal{H}_1, \dots, \mathcal{H}_n$ pravimo sodbe [3, Section 2]. Sodbam $\mathcal{H}_1, \dots, \mathcal{H}_n$ pravimo *premise* in $\mathcal{S}$ je *sklep*. Kasneje bomo videli kako lahko taka pravila sestavljamo skupaj, da dobimo izpeljana pravila. Takim zapisov dokazov pravimo *sklepna drevesa*. Navadno pišemo zraven še kontekst, ki ga ponavadi označimo z Γ . Sodba $\Gamma \vdash A$ pravi, da iz konteksta Γ lahko vzamemo izjavo A in jo uporabimo.

$$\frac{\Gamma \vdash A \quad \Gamma \vdash B}{\Gamma \vdash A \wedge B} \wedge I$$

Zgornje pravilo preberemo kot, če imamo poljubni resnični izjavi A in B , potem v tem kontekstu velja tudi $A \wedge B$.

Za konjunkcijo imamo dva pravila uporabe (ang. elimination rule) — levo in desno.

$$\frac{\Gamma \vdash A \wedge B}{\Gamma \vdash A} \wedge E_L \quad \frac{\Gamma \vdash A \wedge B}{\Gamma \vdash B} \wedge E_D$$

Implikacija. Malce bolj zahtevno pravilo vpeljave ima implikacija. Če lahko iz A izpeljemo B potem velja $A \implies B$. To zapišemo kot

$$\frac{\Gamma, A \vdash B}{\Gamma \vdash A \implies B} \Rightarrow I$$

Torej če lahko iz konteksta Γ , ki mu je pridružena izjava A , izpeljemo izjavo B , potem lahko iz konteksta Γ izpeljemo implikacijo $A \implies B$.

Preden nadaljujemo na primer kako lahko izpeljemo implikacijo, si moramo pogledati eno od osnovnih pravil sklepanja in sicer pravilo aksioma.

$$\frac{}{\Gamma, A \vdash A} Ax$$

To pravilo pravi, da brez dodatnih pogojev lahko sklepamo, da iz konteksta, ki vsebuje A lahko vzamemo A . Pogosto se v literaturi pojavlja tudi notacija

$$\frac{A \in \Gamma}{\Gamma \vdash A} Ax.$$

Sedaj si lahko pogledamo primer dokaza za $A \Rightarrow (B \Rightarrow (A \wedge B))$

$$\frac{\frac{\frac{\frac{\frac{\frac{}{A, B \vdash A} Ax}{A, B \vdash A \wedge B} \wedge I}{A \vdash B \Rightarrow (A \wedge B)} \Rightarrow I}{\vdash A \Rightarrow (B \Rightarrow (A \wedge B))} \Rightarrow I$$

Pri tem primeru bi radi poudarili strukturo *dokaznega drevesa*. Pravila dokazovanja se gradijo v večja *izpeljana pravila* v obliki usmerjenega drevesnega grafa, kjer so vozlišča sodbe in povezave pravila med njimi. Razvidno je tudi, da trditev $A \Rightarrow (B \Rightarrow (A \wedge B))$ drži sama po sebi in nismo potrebovali nobenih premis, da smo to pokazali. Pravilo aksioma deluje kot predpostavka.

Naj bralca ne zmoti dejstvo, da smo označevali pridružitev izjave h kontekstu ter združitev dveh izjav v kontekst z enako oznako “,”. Kontekst namreč predstavlja izjave $A_1, \dots, A_n$, torej notacija Γ, A predstavlja $A_1, \dots, A_n, A$.

Prav tako naj bralca ne zmoti pomankanje konteksta v zadnjem sklepu, namreč Γ lahko predstavlja tudi prazen kontekst, torej smo še vedno upoštevali pravila sklepanja. Kljub temu, da nabor ponavadi v matematiki pomeni urejeno zaporedje, se tukaj zavedajmo, da zaporedje izjav v kontekstu ni pomembno.

Pravilo uporabe za implikacijo je enostavnejše.

$$\frac{\Gamma \vdash A \Rightarrow B \quad \Gamma \vdash A}{\Gamma \vdash B} \Rightarrow E$$

Če vemo da iz A sledi B in vemo A , potem B .

Disjunkcija. Disjunkcija ima levo in desno pravilo vpeljave.

$$\frac{\Gamma \vdash A}{\Gamma \vdash A \vee B} \vee I_L \quad \frac{\Gamma \vdash B}{\Gamma \vdash A \vee B} \vee I_D$$

Ko v dokazu želimo uporabiti disjunkcijo, ne vemo, ali drži leva ali desna stran. Torej ju ločimo in pokažemo, da iz obeh lahko dobimo željen cilj.

$$\frac{\Gamma \vdash A \vee B \quad \Gamma, A \vdash C \quad \Gamma, B \vdash C}{\Gamma \vdash C} \vee E$$

Vidimo da je to pravilo sklepanja podobno pravilu uporabe za implikacijo, saj je spet potrebno pokazazati, da iz A lahko dobimo C , kot da bi vpeljali implikacijo. Poglejmo si primer kako uporabimo disjunkcijo, tako da dokažemo $(A \vee B) \Rightarrow (B \vee A)$. Najprej si samo vizualizirajmo, kaj splo želimo dokazati.

$$\frac{\frac{\frac{\frac{}{A \vee B \vdash A \vee B} Ax}{\vdash (A \vee B) \Rightarrow (B \vee A)} \Rightarrow I$$

Nadaljujemo na edini način, ki nam je na voljo in sicer razdelitev $A \vee B$ na primere.

$$\frac{\frac{\overline{A \vee B \vdash A \vee B} \text{ } Ax \quad \frac{\frac{?}{A \vee B, A \vdash B \vee A} \quad \frac{?}{A \vee B, B \vdash B \vee A}}{A \vee B \vdash B \vee A} \vee E}{\vdash (A \vee B) \implies (B \vee A)} \Rightarrow I$$

Moramo samo še ugotoviti, kaj napisati namesto vprašajev. Lotimo se manjšega podpri-mera.

$$\frac{?}{A \vee B, A \vdash B \vee A}$$

Glede na to, da v sklepu potrebujemo dijunkcijo, lahko poskusimo vpeljati to disjunkcijo.

$$\frac{\frac{?}{A \vee B, A \vdash A}}{A \vee B, A \vdash B \vee A} \vee I_D$$

Da iz konteksta $A \vee B, A$ lahko vzamemo A pa nam pove pravilo hipoteze.

$$\frac{\overline{A \vee B, A \vdash A} \text{ } Ax}{A \vee B, A \vdash B \vee A} \vee I_D$$

Simetrično dobimo dokaz

$$\overline{A \vee B, B \vdash B \vee A}$$

Vstavimo to v naš večji dokaz, da dobimo celotno izpeljavo z osnovnimi pravili.

$$\frac{\overline{A \vee B \vdash A \vee B} \text{ } Ax \quad \frac{\overline{A \vee B, A \vdash A} \text{ } Ax}{A \vee B, A \vdash B \vee A} \vee I_D \quad \frac{\overline{A \vee B, B \vdash B} \text{ } Ax}{A \vee B, B \vdash B \vee A} \vee I_L}{\frac{A \vee B \vdash B \vee A}{\vdash (A \vee B) \implies (B \vee A)} \Rightarrow I} \vee E$$

V kasnejših poglavjih bomo povedali še pravila za eksistenčni in univerzalni kvantifikator ter za negacijo.

3 Curry-Howardov izomorfizem

Matematične dokaze iz logike lahko interpretiramo kot računalniške programe, ali pa kot sklepe v teoriji tipov. Notacija je zelo podobna kot ta, ki smo jo obravnavali v prejšnjem poglavju. Spodaj je primerjava pravil sklepanja iz logike in teorije tipov.

Produktni tip.

$$\frac{\Gamma \vdash A \quad \Gamma \vdash B}{\Gamma \vdash A \wedge B} \wedge I \quad \frac{\Gamma \vdash a : A \quad \Gamma \vdash b : B}{\Gamma \vdash (a, b) : A \times B} \times I$$

$$\frac{\Gamma \vdash A \wedge B}{\Gamma \vdash A} \wedge E_L \quad \frac{\Gamma \vdash x : A \times B}{\Gamma \vdash fst(x) : A} \times E_1$$

$$\frac{\Gamma \vdash A \wedge B}{\Gamma \vdash B} \wedge E_D \quad \frac{\Gamma \vdash x : A \times B}{\Gamma \vdash snd(x) : B} \times E_2$$

Na tej točki je spet potrebno pojasniti nekaj notacije. Kot že rečeno, Γ predstavlja kontekst, torej končen nabor predpostavk. $\Gamma \vdash A$ se imenuje *sekvent* [4, L2.8] in pravi, da iz konteksta Γ lahko vzamemo A , da ga uporabimo naprej. Podobno interpretiramo sekvent tudi v teoriji tipov. Se pravi $\Gamma \vdash a : A$ v kontekstu G lahko uporabimo element a tipa A , kjer je kontekst v tem primeru končen nabor tipov, katerih elemente lahko uporabljamo.

Izpostavimo dejstvo, da nismo povedali “kaj” je pravzaprav tip, le kako ga konstruiramo in uporabljamo. Namreč ni zares pomembno kako “izgleda” posamezen tip, vendar kako ga uporabljamo. Torej na nek način so tipi bolj abstraktna stvar kot izjave. Ampak kljub temu, nam povedo nekaj bolj konkretnega, namreč operiramo z elementi. Pravila sklepanja nam v teoriji tipov pokažejo kako konstruiramo konkretne elemente iz elementov drugih tipov. Torej vemo kako iz a tipa A in iz b tipa B konstruirati element (a, b) tipa $A \times B$. To lahko naredimo, ker v teoriji tipov nimamo le pravil vpeljave in uporabe, temveč tudi pravila računanja. Z njimi lahko dobimo eksplicitne zapise elementov konstruiranih tipov.

Za produktni tip imamo naslednji pravili računanja.

$$fst((a, b)) \equiv a \qquad \qquad \qquad snd((a, b)) \equiv b$$

Funkcijski tip.

$$\begin{array}{c} \frac{\Gamma, A \vdash B}{\Gamma \vdash A \Rightarrow B} \Rightarrow I \qquad \frac{\Gamma, x : A \vdash b : B}{\Gamma \vdash \lambda x : A. b : A \rightarrow B} \rightarrow I \\[10pt] \frac{\Gamma \vdash A \Rightarrow B \quad \Gamma \vdash A}{\Gamma \vdash B} \Rightarrow E \qquad \frac{\Gamma \vdash f : A \rightarrow B \quad \Gamma \vdash a : A}{\Gamma \vdash f a : B} \rightarrow E \end{array}$$

Pri vpeljavi funkcijskega tipa, je pomembna spremenljivka $x : A$. Pkazati moramo, da iz katerega koli elementa $x : A$ lahko dobimo nek element $b : B$, ki je lahko odvisen od $x : A$. Potem lahko skonstruiramo funkcijo, ki vsakemu x -u predpiše pripadajoč b . Predpis $\lambda x : A. b : A \rightarrow B$ je v λ -notaciji. Pove, da vezemo spremenljivko x tipa A in vrnemo izraz b . To je funkcija tipa $A \rightarrow B$.

Pri pravilu uporabe za funkcijo je sklep zapisan z $f a : B$. To je ekvivalentno temu, da bi pisali $f(a)$, vendar v teoriji tipov navadno pišemo funkcije brez oklepajev.

Pravilo računanja za funkcijski tip nam pove, kako se obnaša λ -račun.

$$(\lambda x : A. b)u \equiv b[x/u]$$

Operacijo $b[x/u]$ imenujemo β -*redukcija*, pomeni pa, da vse pojavitve x v b zamenjamo z u , torej kot da bi evalvirali funkcijo $b(x)$ v točki u . Sicer b ni nujno funkcijskega tipa, vendar je vseeno lahko odvisen od x .

Pozorni moramo biti, da ponesreči ne spremenimo pomena elementa $b : B$, namreč $(\lambda x : A. \lambda y : B. x)[y/x] \neq \lambda x : A. \lambda x : B. x$ ¹. Najprej moramo vse vezane spremenljivke preimenovati.

$$(\lambda x : A. \lambda y : B. x)[y/x] = (\lambda z : A. \lambda y : B. z)[y/x] = \lambda z : A. \lambda x : B. z$$

Pozor! Pri tem primer ne evalviramo funkcije v neki točki, temveč preimenujemo vezane spremenljivke. Če bi želeli evalviratu ta labda račun v nekem elementu $y : A$ bi napisali

$$(\lambda x : A. \lambda y : B. x)y = (\lambda y : B. x)[x/y] = (\lambda z : B. x)[x/y] = (\lambda z : B. y).$$

¹Izraz na desni strani neenakosti ni validen, saj dvakrat vezemo isto spremenljivko

Dobili bi torej λ -funkcijo, ki vse elemente iz B slika v $y : A$.

Vsotni tip.

$$\begin{array}{c}
\frac{\Gamma \vdash A}{\Gamma \vdash A \vee B} \vee I_L \qquad \frac{\Gamma \vdash a : A}{\Gamma \vdash \text{in}_1(a) : A + B} +I_L \\
\\
\frac{\Gamma \vdash B}{\Gamma \vdash A \vee B} \vee I_D \qquad \frac{\Gamma \vdash b : B}{\Gamma \vdash \text{in}_2(b) : A + B} +I_D \\
\\
\frac{\Gamma \vdash A \vee B \quad \Gamma, A \vdash C \quad \Gamma \vdash C}{\Gamma \vdash C} \vee E \\
\\
\frac{\Gamma \vdash c : A + B \quad \Gamma, x : A \vdash a : C \quad \Gamma, y : B \vdash b : C}{\Gamma \vdash (\text{case } c \text{ of } \text{in}_1(x) \mapsto a \mid \text{in}_2(y) \mapsto b) : C} +E
\end{array}$$

Tako kot pri funkcijskem tipu, moramo najti način, da za vsak element $x : A$ lahko dobimo $a : C$ in za vsak element $y : B$ najdemo $b : C$. To nam zagotovi, da lahko za element $c : A + B$ najdemo nek element tipa C .

Za vsotni tip imamo dva računska pravila.

$$\begin{array}{l}
\text{case } \text{in}_1(s) \text{ of } \text{in}_1(x) \mapsto t \mid \text{in}_2(y) \mapsto r \equiv t[x/s] \\
\text{case } \text{in}_2(s) \text{ of } \text{in}_1(x) \mapsto t \mid \text{in}_2(y) \mapsto r \equiv r[y/s]
\end{array}$$

4 Linearna naravna dedukcija

Sedaj lahko predstavimo linearno logiko. Glavna lastnost v linearni logiki je, da vsako premiso uporabimo natanko enkrat. Na prvi pogled se to zdi nenavadno, saj si resnico predstavljamo, kot neodvisno od tega ali smo jo že uporabili ali ne. Vendar videli bomo, da linearna logika v veliko primerih bolje opiše svet.

Recimo, da stojimo pred avtomatom. Avtomat sprejema samo kovance za 50c in 1€ in potrebno je vstaviti točen znesek. Na avtomatu si lahko kupimo čokoladico za 1€, bombončke za 50c ali pa sladko pijačo za 1€. To lahko predstavimo s pravili sklepanja.

$$\begin{array}{ccc}
\frac{1\text{€}}{\text{pijača}} & \frac{1\text{€}}{\text{čokoladica}} & \frac{50\text{c}}{\text{bombončki}}
\end{array}$$

Recimo, da imamo tri kovance za 1€ in štiri kovance za 50c. Skupina naslednjih sklepov je veljavna v linearni logiki, če jih naredimo skupaj.

$$\begin{array}{ccccc}
\frac{1\text{€}}{\text{pijača}} & \frac{50\text{c}}{\text{čokoladica}} & \frac{1\text{€}}{\text{čokoladica}} & \frac{50\text{c}}{\text{čokoladica}} & \frac{50\text{c}}{\text{bombončki}} & \frac{1,5\text{€}}{1,5\text{€}}
\end{array}$$

Vidimo, da se vrednost ne izgubi. Začeli smo z 5€ in dobili ven stvari v vrednosti 5€. Ne moremo ničesar izgubiti in prav tako ne moremo ničesar pridobiti. Istega kovanca ne moremo uporabiti dvakrat, torej so sklepi, ki jih imamo na voljo odvisni od stanja virov in vsakič ko uporabimo sklep spremenimo stanje virov.

Hkratna konjunkcija. Recimo, da lahko s pomočjo konteksta Γ pokažemo izjavo A in s pomočjo konteksta Δ pokažemo izjavo B , Potem lahko s kontekstom Γ, Δ pokažemo da velja A in B . Oznaka za tako konjunkcijo v linearni logiki je tenzor $A \otimes B$.

$$\frac{\Gamma \vdash A \quad \Delta \vdash B}{\Gamma, \Delta \vdash A \otimes B} \otimes I$$

Glede na to, da $\otimes$ predstavlja konjunkcijo, bi lahko zmotno mislili, da je pravilo uporabe za $\otimes$, enako kot za $\wedge$, vendar pravilo za uporabo $\wedge$ izgubi del informacije.

$$\frac{A \wedge B}{A} \wedge E_L$$

Na vrhu imamo A in tudi B , vendar v sklepu uporabimo samo A . Ne seldimo pravilom linearne logike, ki pravi, da je potrebno vsako premiso uporabiti natanko enkrat. Pravilo vpeljave hkratne konjunkcije pravi, če lahko dosežemo cilj C s pomočjo virov A in B in če imamo $A \otimes B$, lahko dosežemo cilj C .

$$\frac{\Gamma \vdash A \otimes B \quad \Delta, A, B \vdash C}{\Gamma, \Delta \vdash C} \otimes E$$

Na tej točki je smiselno omeniti, kaj v linearni logiki pomeni kontekst. To je tenzorski produkt *virov* $A_1 \otimes \dots \otimes A_n = \Gamma$. Navadno tenzorjev ne pišemo in vire preprosto ločimo z vejico $A_1, \dots, A_n$. Torej to nam pove, da morajo viri v kontekstu biti resnični oziroma na voljo v istem stanju.

Alternativna konjunkcija. Recimo da imamo 1€ . Potem sli lahko kupimo ali čokoladico ali pijačo, vendar ne oboje na enkrat.

$$\frac{\Gamma \vdash A \quad \Gamma \vdash B}{\Gamma \vdash A \& B} \& I$$

To pravilo vpeljave izgleda enako kot pravilo vpeljave za konjunkcijo $\wedge$, vendar pomen je drugačen, saj $A \& B$ predstavlja "natanko eno od A in B ".

Če imamo vire za $A \& B$, vemo da imamo vire za natanko eno od A in B . Torej pravili uporabe izgledata takole:

$$\frac{\Gamma \vdash A \& B}{\Gamma \vdash A} \& E_L \qquad \frac{\Gamma \vdash A \& B}{\Gamma \vdash B} \& E_D$$

Linearna implikacija. [1, poglavje 1.2] Za linearno implikacijo se uporablja znak $\multimap$ imenovan *lizika* [4, L20.3].

$$\frac{\Gamma, A \vdash B}{\Gamma \vdash A \multimap B} \multimap I.$$

Linearna implikacija $A \multimap B$ porabi vir A , vendar implikacijo sama ostane, torej če imamo na voljo še kakšen vir A , ga lahko ponovno uporabimo.

$$\frac{\Gamma \vdash A \quad \Delta \vdash A \multimap B}{\Gamma, \Delta \vdash B}$$

Pravilo uporabe za linearno implikacijo upošteva, da se viri, ki jih uporabimo, da dobimo A in viri, da dobimo $A \multimap B$ lahko prekrivajo.

Sedaj lahko pokažemo eno pomembno pravilo imenovano *rez*. Intuitivno se nam zdi, da če z viri Γ, A lahko dosežemo B in z viri Γ lahko dosežemo A , potem bi viri Γ zadostovali, da dosežemo B .

$$\frac{\Gamma \vdash A \quad \Gamma, A \vdash B}{\Gamma \vdash B} \text{ cut}$$

In res s pravili iz poglavja Naravna dedukcija lahko izpeljemo rez.

$$\frac{\Gamma \vdash A \quad \frac{\Gamma, A \vdash B}{\Gamma \vdash A \implies B} \Rightarrow I}{\Gamma \vdash B} \Rightarrow E$$

V linearni logiki je to pravilo rahlo drugačno, saj pri $\Rightarrow I$ uporabimo dvakrat isti kontekst, medtem ko pri $\multimap I$ tega ne smemo narediti.

$$\frac{\Gamma \vdash A \quad \frac{\Delta, A \vdash B}{\Delta \vdash A \multimap B} \multimap I}{\Gamma, \Delta \vdash B} \multimap I$$

Izpeljano pravilo je torej

$$\frac{\Gamma \vdash A \quad \Delta, A \vdash B}{\Gamma, \Delta \vdash B} \text{cut.}$$

5 Beseda avtorja

Rada bi se opravičila svojemu mentorju, Matiju Pretnarju, za osnutek, ki presega pet vsebinskih strani. Prv tako bi se rada opravičila za vse pravopisne in slogovne napake, še posebej za vse narobe postavljene vejice. Hkrati pa bi ga prosila, če lahko komentira uporabo citatov v tem diplomskem delu. Ali je citiranje iz kje dobim eno besedo, ki jo prevedem iz angleščine preveč? Prav tako me zanima, ali v kakšnih odsekih citiram premalo. Par citatov manjka (2), saj $\text{\LaTeX}$ ne najde stvari, ki jih mora v moji `bib` datoteki.

Prav tako bi želela od svojega mentorja, da mi pove, če se mu zdijo kakšni necitirani podatki neznani ali celo nepravilni, da preverim, če resnično držijo, ker mi je nekaj podatkov povedala asistentka Luna Strah in jih nisem (še) preverila. Delovala sem pod sklepom, da ko preberem več o linearni logiki, bom naletela na primernejše razlage, ki bi jim morda lahko rekli “citirable”, če ne bi bili profesionalni.

Cenila bi tudi komentar o mojih prevodih, predvsem o prevodu “computation rule” (iz poglavja Curry-Howardov izomorfizem v zadnjem in predzadnjem odstavku). Iz Simpsonovih predavanj iz Logike v računalništvu sem izvedela, kaj so computation rules, vendar se mi zdi pravilo računanja grd prevod. Asistenka Strah je omenila še eno ime za ta pravila, vendar sem si jih nisem uspela zapomniti.

6 Kako nadaljevati diplomsko delo

- Pri poglavju Naravna dedukcija ali kasneje bi rada še opisala pravila za univerzalni in eksistenčni kvantifikator, negacijo, resnico in neresnico. Potem bi primerjala pravila klasične in konstruktivne logike.
- Trenutno na treh različnih mestih povem kaj je kontekst in to bi rada spremenila.
- Dodala bom še ostala osnovna pravila v linearni logiki.
- Nato bom povezala kje v programskih jezikih se uporabljajo linearni tipi in zakaj.
- Mogoče bom naredila in opisala minimalni programski jezik, ki vsebuje vse ekvivalente veznikov iz linearne logike.

7 Slovarček

judgment sodba

premise premisa

conclusion sklep

inference rule pravilo sklepanja

computation rule pravilo računanja

β -reduction β -redukcija

resource vir

simultaneous conjunction hkratna konjunkcija

alternative conjunction alternativna konjunkcija

cut rez

loli lizika

external choice zunanja izbira

Literatura

- [1] Andrew G Barber. Linear type theories, semantics and action calculi. 1997.
- [2] Jean-Yves Girard. Linear logic. *Theoretical computer science*, 50(1):1–101, 1987.
- [3] Frank Pfenning. Lecture notes on natural deduction. *Course CMU*, 15317, 2009.
- [4] Frank Pfenning. Lecture notes on truth is ephemeral. 2023.